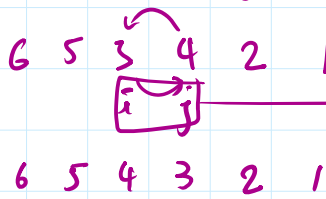
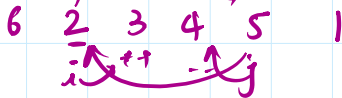


Reverse an array

①

arr:



space complex

$O(1)$

time complexity

$O(n)$

for

while

$(i \leq j)$ [not swapped]

swap(— —);

$i++$
 $j--$

}

②

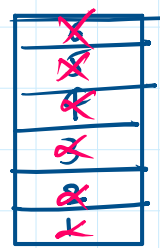
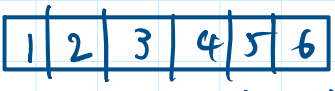
(stacks)

DS

LIFO

Last in first out

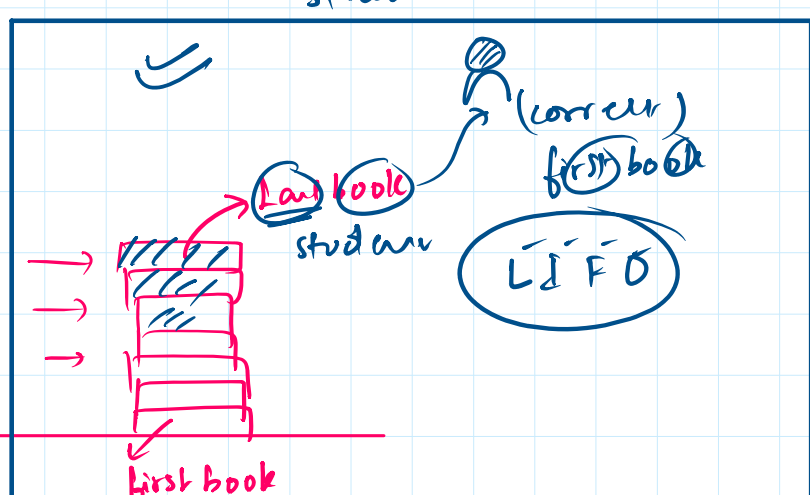
functions: pop(),
peek(),
top(),
push()



vertically

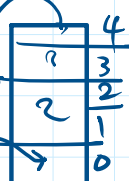
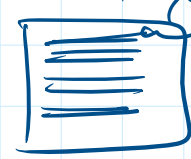
for:

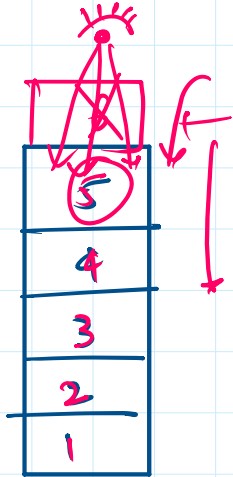
stack



GitHub

stash



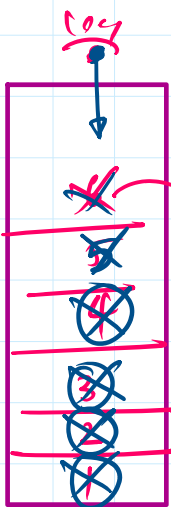


* push
 $\Rightarrow st.push(6);$
 * pop() $\rightarrow$ remove
 $st.pop();$
 * top();
 $st.top(); \rightarrow op = 5$

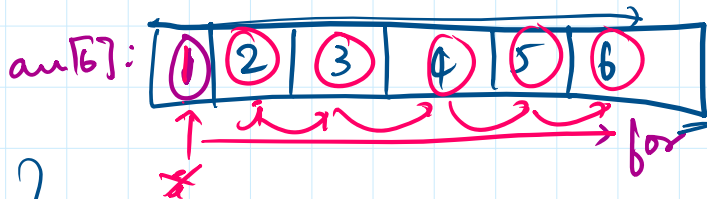
$stack<int> st$
 $\downarrow$
 data type

* Reverse an Array using stack;

stack

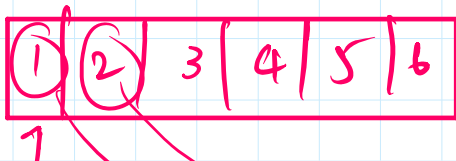


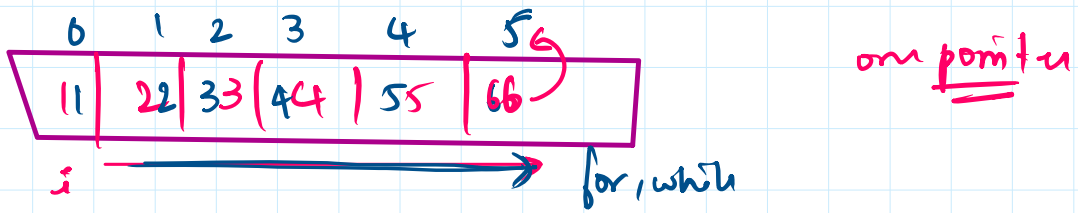
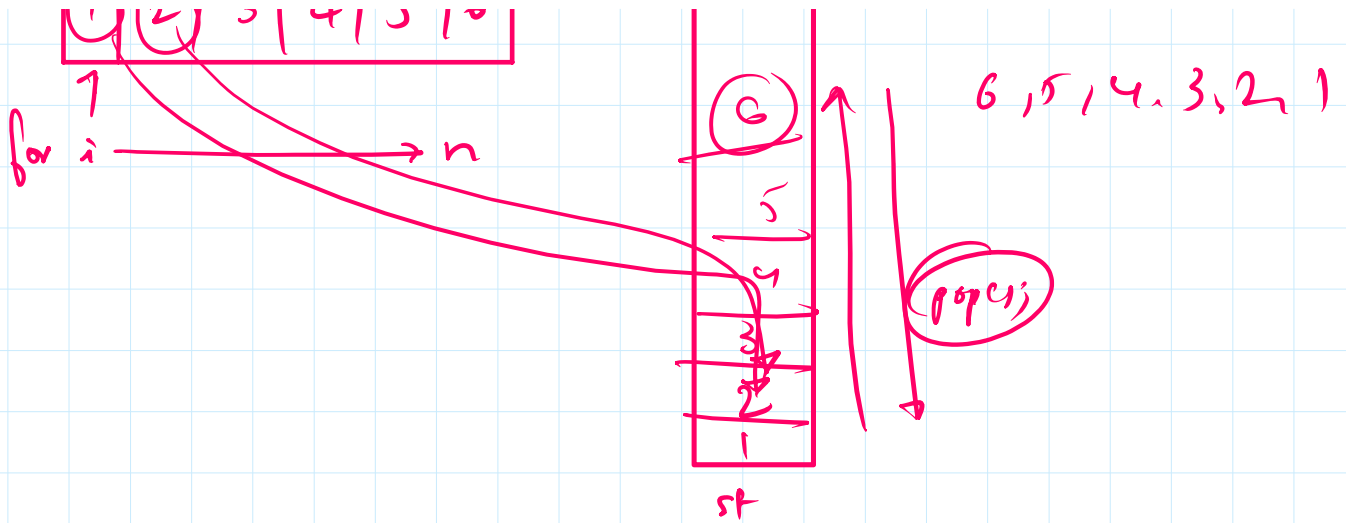
$stack<int> st$



$cout << st.top();$
 $st.pop();$
 $\rightarrow$ print : 6 5 4 3 2 1 Reverse

stack = space $\uparrow \uparrow \uparrow$ $O(n^2) + O(n)$
 $\rightarrow$ $O(2n)$





key = 66

o/p = 5

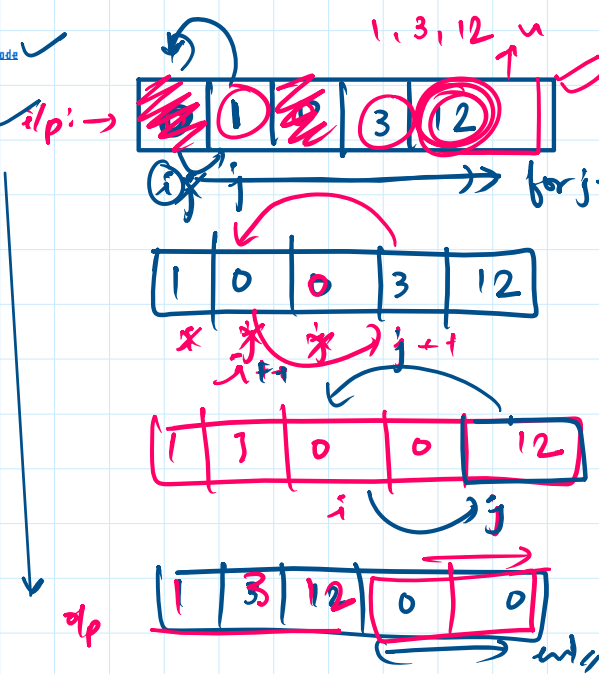
linear search → $O(n)$

binary search → $O(\log n)$

time complexity

mathematical proof

4 for



1 1 1 2 1 1

$O(n^2)$

trick

$O(2^n) > O(n^2) > O(n) > O(\log n) > O(1)$

two stars

time complexity optimize of code

time comp

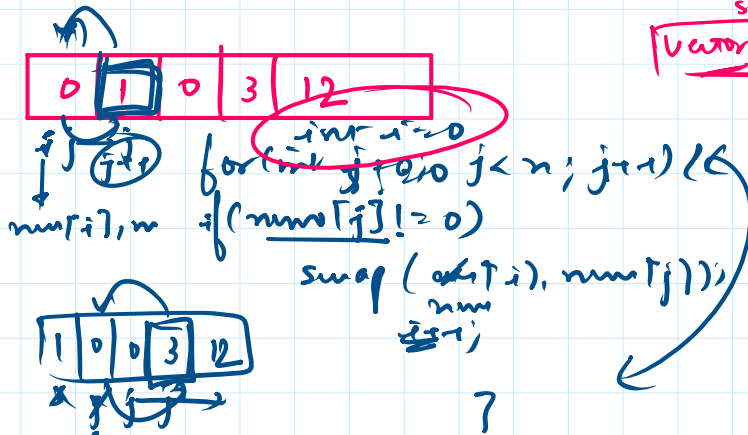


int arr[3]

size

vector<int> arr = {1, 2, 3}

arr



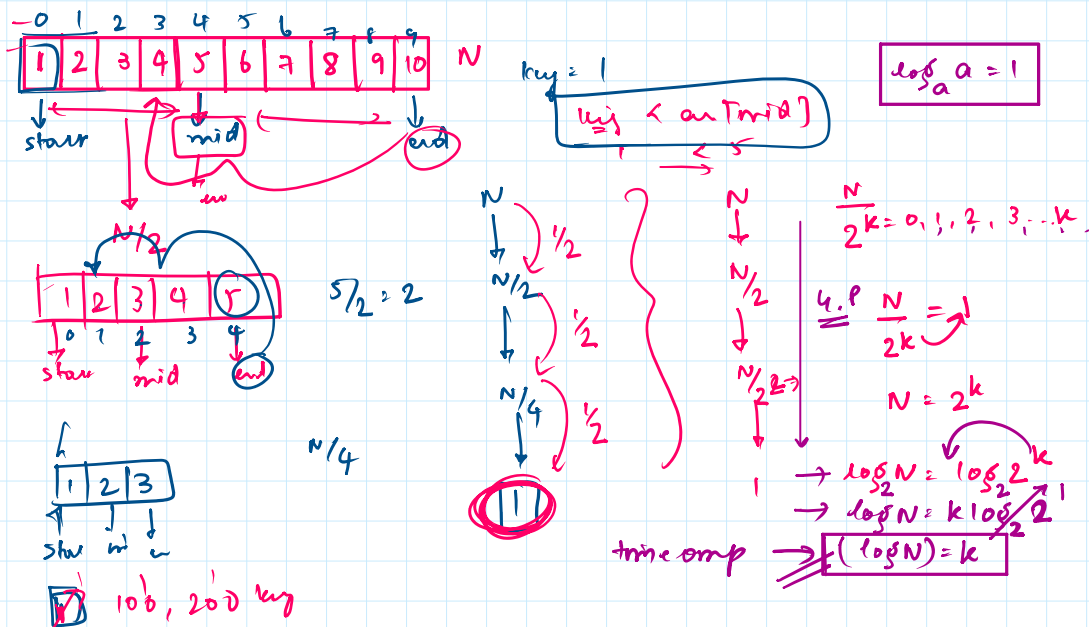
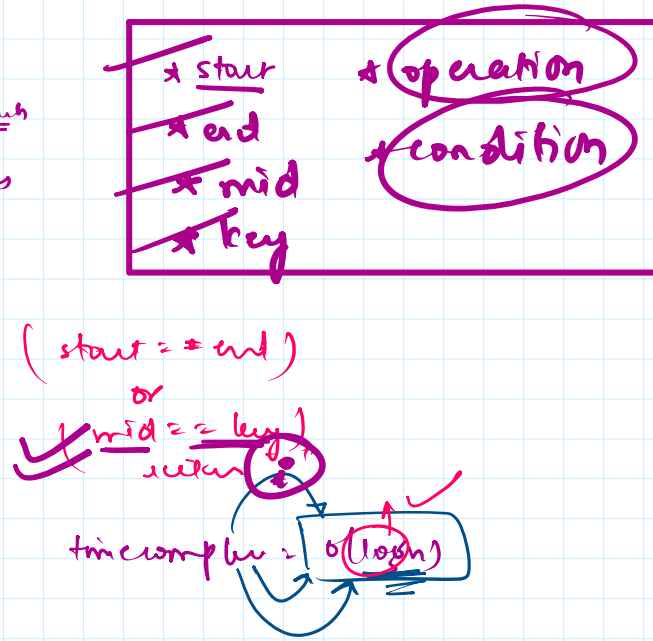
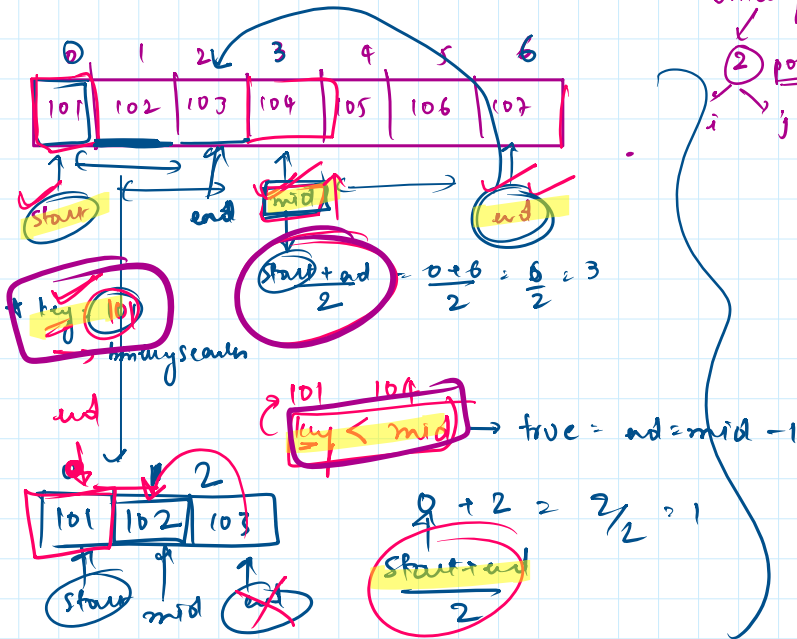
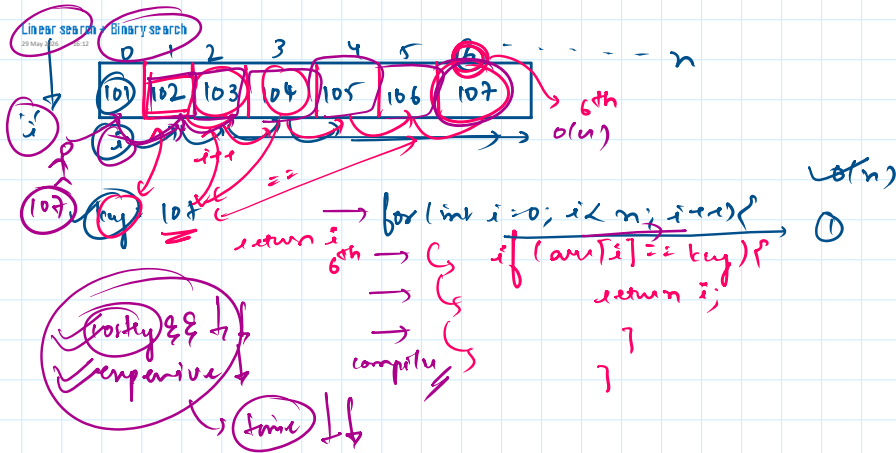
$O(n)$

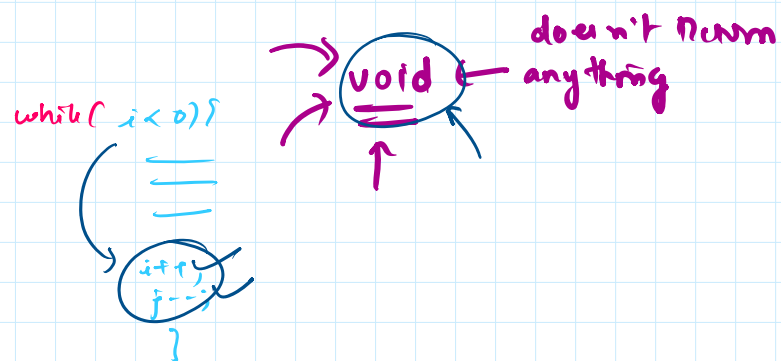
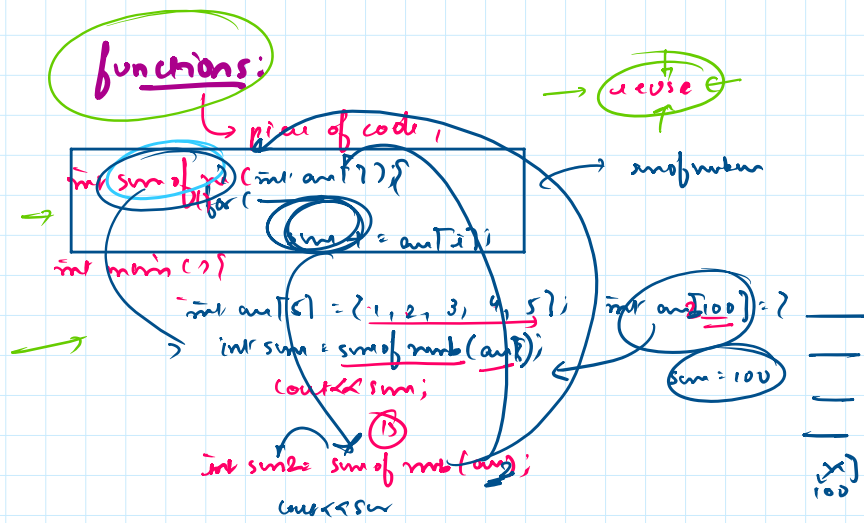
$O(n^2)$

for (int i = 0; i < n; i++)

for (j = i; j < n; j++)

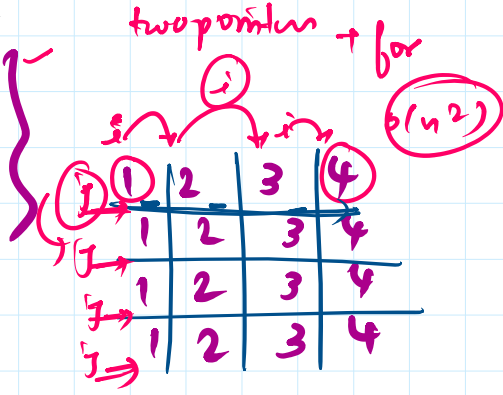
$O(n^2) \rightarrow O(n)$





→ recursion.
* function calling itself.

Q.1
1234
1234
1234
1234



int i=0;

while (i < 4) {

int j=0;

while (j < 4) {

cout << " ";

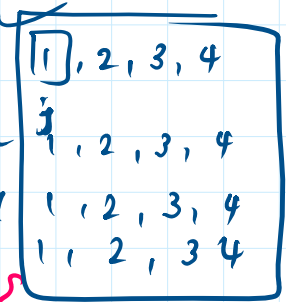
j++

cout << endl;

i++

}

}



1111
2222
3333
4444

(4)

int i=1;

while (i <= 4) {

int j=1;

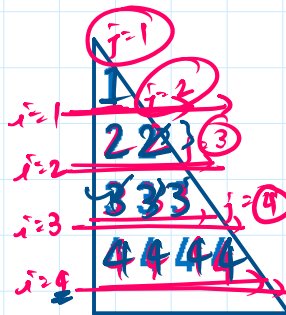
while (j <= i) {

int i=0;

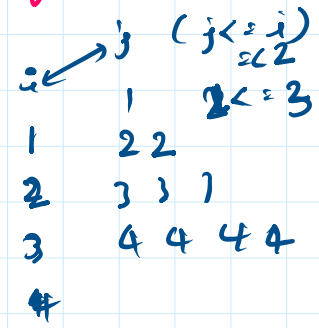
while (i < 4) {

int j=1;

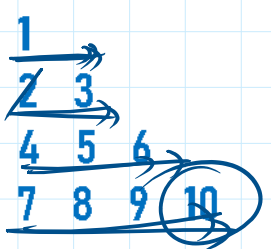
while (j <= i) {



no of row (i) = count
no of col (j)



1, 2, 3, 4
1, 2, 3, 4
1, 2, 3, 4
1, 2, 3, 4

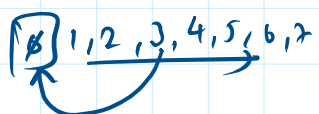


1 2 3, 4, 5, 6

count → for j while j

int i=0;

int count=0



1 0 7 11

0 1 2
0
0 1
0 1 2
0 1 2 3

